Home
Library
Profile
Stories
Stats

Following
Dev Genius
Python in Plain En...
Generative AI
The Useful Tech
Level Up Coding
Top Python Librari...
Realworld AI Use C...
Deep concept
AI Mind
The Mac Alchemist

CodeToDeploy

Member-only story

CLAUDE.md for .NET 10: Turn Claude From Autocomplete Into a Teammate

Abe Jaber Follow 5 min read · Jan 27, 2026

Claude isn't "bad at .NET".

Claude is **blind**.

You ask it to "add an endpoint", and it confidently:

- invents folder structure that doesn't match your repo
- adds patterns you don't use
- skips your build/test workflow
- breaks your conventions and forces you into cleanup mode

That's not an AI problem. It's an onboarding problem.

The fix is a single file that becomes your **project's operating system** for Claude:

`CLAUDE.md`.

In Claude Code, memory files like `./CLAUDE.md` (and modular rules in `./.claude/rules/*.md`) are automatically loaded as context when you launch the tool, so Claude starts every session with your project's map + rules instead of guessing.

This article gives you:

- the **minimum** CLAUDE.md that actually moves the needle for .NET teams
- a **production-ready template** you can paste today
- a rule system that prevents Claude from "framework cosplay" and costly rework

To Start!

. . .

What CLAUDE.md actually does (in plain English)

Think of CLAUDE.md as **onboarding notes for a new senior dev.**

Except this "dev" will:

- follow your instructions consistently
- remember them across sessions
- stop re-inventing your architecture every time

Claude's docs describe CLAUDE.md as a project configuration file that lives in your repo and gets loaded into context automatically.

And importantly: **you don't have to start from scratch.** Claude Code supports `/init` to generate a starter CLAUDE.md by scanning your repo—then you refine it.

. . .

The rule: your CLAUDE.md should prevent expensive mistakes

If CLAUDE.md doesn't prevent one of these, it's not doing its job:

- **wrong patterns** (repositories, AutoMapper, random abstractions you don't use)
- **wrong workflow** (changes without plan, no tests, no validation)
- **wrong defaults** (timeouts, cancellation, retries, background work, DI lifetimes)

So don't write it like documentation.

Write it like: **guardrails + map + commands.**

. . .

Claude Code memory layout (so you don't fight it later)

Claude Code supports a small hierarchy of memory files, including:

- `./CLAUDE.md` for team-shared project memory
- `./.claude/rules/*.md` for modular rules
- `~/.claude/CLAUDE.md` for your personal global preferences
- `./CLAUDE.local.md` for your private project notes (and it's designed not to be committed)

That means you can:

- keep the root CLAUDE.md **short**
- push detailed rules into `.claude/rules/`
- keep your personal stuff in `CLAUDE.local.md` without polluting the repo

. . .

The Minimum Viable CLAUDE.md for a .NET 10 backend

Create `CLAUDE.md` in your repo root:

```
# CLAUDE.md — Project Instructions (.NET)
## What this repo is
- .NET 10 backend API focused on scalability, p95/p99 latency, and producti
- Prefer simple, copy/pasteable patterns over "clever architecture".

## Tech stack (depending on your setup)
- .NET 10 / C# (modern style)
- ASP.NET Core (Minimal APIs)
- Dapper (NOT EF Core)
- Redis (IDistributedCache / StackExchange.Redis depending on project)
- Azure hosting (App Service / Containers)
- Observability: Application Insights (logs + metrics)

## Repo map (edit to match your folders)
- src/Api/        → endpoints, DI, middleware, startup
- src/App/        → use-cases/services (business orchestration)
- src/Infra/      → DB, Redis, HTTP clients, external integrations
- tests/          → unit + integration tests

## Hard rules (do not violate)
- NEVER add new framework layers or "Clean Architecture cosplay".
- NEVER introduce patterns we don't use (AutoMapper, repositories, magic abs
- Always pass CancellationToken through all async calls.
- No sync-over-async (no .Result/.Wait).
- No Task.Run inside request handlers.
- Outbound HTTP MUST have timeouts + cancellation.
- Caching MUST have: time budget, stampede protection strategy, and key vers

## Default workflow
1) Ask for missing requirements before changing code.
2) Propose a plan + list files to touch.
3) Implement smallest change that works.
4) Add/update tests when relevant.
5) Provide commands to verify (build/test/run).

## Commands (edit to match your codebase)
- Build: `dotnet build`
- Test: `dotnet test`
- Run API: `dotnet run --project src/Api`
- Format: `dotnet format`

## Output format
- Prefer short sections, small code blocks, and explain trade-offs.
- When making changes: show diff-level guidance + why.
```

This is enough to stop 80% of Claude's "guessing".

Now let's make it **dangerously effective**.

. . .

Modular rules: stop repeating yourself (and stop Claude from improvising)

Claude Code supports modular rules under `.claude/rules/*.md`.

Create:

`.claude/rules/api-conventions.md`

```
# API conventions (Minimal APIs) ---> Depedning on your setup
- Prefer Minimal APIs (`MapGet/MapPost`) over controllers.
- Endpoints must accept CancellationToken.
- Validate inputs early. Return ProblemDetails for errors.
- For outbound calls: use IHttpClientFactory + timeouts.
- For "slow work": queue it (don't run it in the request).
```

`.claude/rules/perf-reliability.md`

```
# Perf + reliability rules
- All outbound HTTP:
  - Timeout per request
  - CancellationToken wired through
  - Retry ONLY for safe idempotent calls (and never infinite)
- Redis:
  - Cache calls get a small time budget (fail fast)
  - Stampede protection for hot keys (singleflight or equivalent)
  - TTL jitter for large fleets
- Logging:
  - Avoid high-cardinality fields
  - Avoid expensive structured logs in hot paths
```

`.claude/rules/security.md`

```
# Security rules
- No secrets in code or CLAUDE.md.
- Avoid putting sensitive IDs/tokens in query strings.
- Auth must be explicit on endpoints (no accidental public routes).
- CORS must be restrictive (no AllowAnyOrigin in prod configs).
```

Now your root CLAUDE.md stays short and readable, and your rules are targeted.

. . .

The “make Claude useful fast” loop (how you actually maintain this)

Claude’s own guidance is basically: treat CLAUDE.md like a living config you refine over time.

Here’s the loop I recommend:

1. Run `/init` once to get a starter file.
2. Delete anything generic. Keep only repo-specific truth.
3. Every time Claude does something annoying, add a rule that prevents it.
4. Move big rule chunks into `.claude/rules/` so the root stays clean.
5. When you switch tasks, clear context with `/clear` so Claude doesn’t drag old irrelevant details into the next feature.

Claude’s blog also mentions using the `#` key to add instructions you keep repeating (so they accumulate into CLAUDE.md over time).

. . .

What to include (and what not to)

Include (high ROI)

- exact tech versions + frameworks you *actually* use
- folder map + “where things go”
- your non-negotiables (timeouts, cancellation, no Task.Run, no sync-over-async)
- build/test/run commands
- patterns you **don’t** want suggested

Don’t include (low ROI or dangerous)

- secrets, tokens, connection strings (ever)
- style rules your formatter already enforces
- huge walls of documentation Claude can look up anyway

Your goal isn’t to teach Claude .NET.
Your goal is to teach Claude **your repo**.

. . .

A simple “test prompt” to validate your CLAUDE.md

After you add CLAUDE.md, run this test prompt:

> “Add a new endpoint that calls a downstream HTTP service and caches the response safely. Show the plan first, then implement.”

If Claude (ex):

- uses Minimal APIs
- wires CancellationToken through
- sets a request timeout
- avoids Task.Run
- adds safe caching patterns

…your CLAUDE.md is working.

If it improvises, your CLAUDE.md is missing rules.

. . .

Wrap-up

If you’re serious about AI-assisted development in .NET, stop prompting harder and start onboarding better.

CLAUDE.md is how you turn Claude from “smart autocomplete” into “consistent teammate.”

Your next move:

1. add the minimum CLAUDE.md from this post
2. create 2–3 modular rule files (`api`, `perf`, `security`)
3. run `/init`, trim it, then iterate